

Cryptography Foundations

Encryption Is Not Magic. It's Math + Trust.

Cryptography provides:

- Confidentiality – encryption hides data
- Integrity – hashing & MACs detect tampering
- Authentication – digital signatures prove identity
- Non-repudiation – signatures prevent denial

Encoding vs Encryption vs Hashing (Know the difference!)

Name	Reversible?	With key?	Example
Encoding	Yes	No key	Base64, URL-encoding
Encryption	Yes	Needs key	AES, RSA
Hashing	No	–	SHA-256

Encoding is NOT security. Base64 of a password is still readable by anyone who decodes it. Malware and DLP tools rely on people confusing these.

Symmetric Encryption: Same Key Both Ways

```
plaintext --[AES(key)]--> ciphertext --[AES(key)]--> plaintext
                        SAME KEY
```

- Fast, good for bulk data (AES-128/256, ChaCha20)
- Problem: key distribution – both sides must share the secret securely

Asymmetric Encryption: Key Pairs

```
plaintext --[RSA(publicKey)]--> ciphertext --[RSA(privateKey)]--> plaintext
```

Everything encrypted with the public key only decrypts with the private key

Solves key distribution (public key = share freely)

Slow – used for key exchange + signatures, not bulk data

How They Combine: Hybrid Crypto

Practical systems use both:

Asymmetric (RSA/ECC) to exchange a **session key**

Symmetric (AES/ChaCha) to encrypt the data with that key quickly

This is how TLS actually works.

The Threat Model

Brute force – guessing the key (kept impractical by key length)

Ciphertext attacks – analyzing ciphertext patterns (prevented by proper modes/IVs)

Side-channel – timing, power, cache leaks of the KEY

Protocol misuse – using crypto wrong (ECB mode, short key, reusing nonces)

Golden rule: cryptography does NOT fix broken systems. A bank vault door hangs on the wall it's attached to. Misused crypto is worse than no crypto – it invites false confidence.

[← Module start](#)

[Hashing, MACs & Data Integrity...](#)